

1º Ano - Licenciatura em Engenharia Informática-EINF2 T1

PROGRAMAÇÃO I

Linguagem C

Lista de Exercícios 3

Docente: Vasconcelos Oliveira
vasconcelos.oliveira@isptec.co.ao
Departamento de Engenharias e Tecnologias - DET
20 de Maio de 2026



Exercício 1: Soma, Média, Quantidade de pares e ímpares

Crie um programa que leia 10 números inteiros e imprima o vetor na ordem inversa. De seguida calcule, a soma, a média e informe quantos elementos são pares e ímpares.

Exercício 2: Valores positivos e negativos

Escrever um programa que pega a dimensão $N = 10$ de um array de **int** preenche o array com valores digitados e o imprima. Em seguida, copie todos os componentes estritamente positivos em uma segunda matriz **Tpos** e todos os valores estritamente negativos em uma matriz **Tneg**. Imprimir **Tpos** e **Tneg**.

Exercício 3: Matriz Transposta

Crie uma matriz 3x3 e gere sua matriz transposta.

Exercício 4: Média de cada linha

Calcule a média dos elementos para cada linha de uma matriz.

Exercício 5: Multiplicação de 2 matrizes

Escreva um programa que realize o produto de duas matrizes quadradas de mesma dimensão.

Exercício 6: Ordenação por seleção (Selection Sort)

Escrever um algoritmo para ordenar um vetor de números inteiros por ordem crescente utilizando a **Ordenação por seleção (Selection Sort)** : Esta ordenação consiste em colocar cada elemento da matriz na sua posição definitiva, do mais pequeno para o maior:

1. Selecionar a primeira posição do vetor.
2. Assumir que o elemento da posição atual é o menor.
3. Comparar esse elemento com todos os elementos à direita.
4. Se encontrar um elemento menor: atualizar a posição do menor elemento.
5. Após terminar as comparações: trocar o menor elemento encontrado com o elemento da posição atual.
6. Avançar para a próxima posição do vetor.
7. Repetir o processo até o penúltimo elemento.

Exercício 7: Ordenação por inserção (Insertion Sort)⁶

Escrever um algoritmo para ordenar um vetor de números inteiros por ordem crescente utilizando a **Ordenação por inserção (Insertion Sort)**: Esta ordenação consiste em construir a sequência ordenada passo a passo, inserindo cada elemento na sua posição correta, do mais pequeno para o maior:

1. Considerar o primeiro elemento como já ordenado.
2. Selecionar o próximo elemento do vetor.
3. Guardar o valor do elemento selecionado.
4. Comparar o elemento guardado com os elementos anteriores.
5. Enquanto o elemento anterior for maior: mover o elemento anterior uma posição à direita.
6. Inserir o elemento guardado na posição correta.
7. Avançar para o próximo elemento do vetor.
8. Repetir o processo até o último elemento.

Exercício 8: Ordenação por Combo (Comb Sort)

Escrever um algoritmo para ordenar um vetor de números inteiros por ordem crescente utilizando a **Ordenação por Combo (Comb Sort)**: Esta ordenação consiste em construir a sequência ordenada passo a passo, inserindo cada elemento na sua posição correta, do mais pequeno para o maior:

1. Definir o intervalo inicial com o tamanho do vector;
2. Definir a constante de redução: factor de redução 1.3 (aproximadamente)
3. Repetir enquanto intervalo > 1 ou houve_trocas = verdadeiro:
 - Atualizar o intervalo: $\text{intervalo} = (\text{intervalo} / \text{factor de redução})$
 - Se intervalo < 1: intervalo = 1
 - Definir variável de controle: houve_trocas = falso
 - Percorrer o vetor de 0 até (n - intervalo - 1):
Se vetor[i] > vetor[i + intervalo] então
trocar os elementos
houve_trocas = verdadeiro.
 - Quando intervalo = 1 e não houver trocas: o vetor está ordenado.

Exercício 9: Pesquisa Dicotômica

Dada um vector ordenada de números reais, escreva um programa que procure um elemento na matriz usando os seguintes métodos: de pesquisa dicotômica.

- O método de **pesquisa sequencial (ou linear)**: consiste em comparar o valor procurado com todos os valores presentes na tabela.
- O método de **pesquisa dicotômica** consiste em:
 1. Determinar **m**, elemento na posição central do array;
 2. Se este for o valor desejado, paramos com sucesso;
 3. Caso contrário, dois casos são possíveis:
 - (a) Se **m** for maior que o valor procurado, já que o array está ordenado, significa que é suficiente continuar a busca na primeira metade do array;
 - (b) Caso contrário, basta pesquisar a metade direita.
 4. Repetimos isso até encontrar o valor pesquisado ou reduza o intervalo de pesquisa para um intervalo vazio, o que significa que o valor pesquisado não está presente.

Exercício 10: Soma de valores do array

Escreva um programa em C que preencha um vector de inteiros e calcule a soma de seus elementos usando um ponteiro para sua varredura.

Exercício 11: Ocorrência de um valor

Escreva um programa que, dado um array de inteiros já inicializado, pergunte ao usuário qual inteiro procurar e então imprima o número de ocorrências desse inteiro no array.

- Declare o array com [].
- Use ponteiros para acessar os elementos do array, ou seja, sem usar uma variável de índice.

Exercício 12: Aritmética de ponteiros

Seja **P** um ponteiro que 'aponta' para um array A:

```
int A [] = {12, 23, 34, 45, 56, 67, 78, 89, 90};
```

```
int * P;
```

```
P = A;
```

Quais valores ou endereços fornecem essas expressões:

- $*P + 2$
- $*(P + 2)$
- $\&P + 1$
- $\&A[4] - 3$
- $A + 3$
- $\&A[7] - P$
- $P + (*P - 10)$
- $*(P + *(P + 8) - A[7])$